Agent Postmortem Generator

Part 14 - Future Roadmap & Extension Strategy

Module Design Document

Purpose: Define how the project should expand after MVP without breaking the deterministic, local-first core.

Project name	Agent Postmortem Generator
Part number	14
Module name	Future Roadmap & Extension Strategy
One-line purpose	Specify the sequencing, guardrails, and decision criteria for post-MVP extensions.

1. Executive Overview

This module defines how Agent Postmortem Generator evolves after the MVP without losing its core identity. It is not a feature backlog. It is a governance document for growth.

The module exists because early roadmap choices are sticky. If the project expands without rules, the result will be the usual software landfill: adapters for everything, half-considered UI features, and analysis logic leaking into presentation layers. This document prevents that.

It fits above all implementation modules and below the project constitution documents. The roadmap informs future module proposals, release sequencing, deprecation strategy, and “do not build yet” decisions.

2. Problem Statement

The exact problem is deciding what to build next after the MVP proves the core postmortem workflow works. The project needs a stable mechanism to evaluate candidate features, sequence them, and reject attractive but premature work.

This is hard because AI tooling projects attract feature sprawl. Once the core pipeline works, every adjacent idea sounds reasonable: more adapters, browser extensions, online sync, richer graphs, LLM-assisted prose, collaborative review, and hosted services. Most of those can quietly destroy the properties that made the MVP trustworthy.

The MVP needs this module so the team can move deliberately from a narrow local tool to a durable product line without breaking the canonical trace model, deterministic analysis, or local-first execution.

3. Goals

Preserve the project’s core contract: everything after normalization remains framework-agnostic, deterministic where possible, and locally executable by default.

Provide a practical sequencing model for future work across adapters, outputs, UX, analysis depth, and ecosystem integrations.

Define release gates so each extension must justify itself against reliability, maintainability, and user value.

Create a shared vocabulary for roadmap decisions that engineers, maintainers, and contributors can use without debating basic boundaries every week.

4. Non-Goals

This module does not define implementation details for any future feature. It defines when and why such a feature may exist.

It does not pre-approve hosted services, telemetry, or collaborative workflows. Those are later questions with separate risk analysis.

It does not override the engineering principles document. If a roadmap item conflicts with the constitution, the item loses.

It does not become a wish list for contributors. “Would be cool” is not a criterion.

5. Responsibilities

Own the long-term expansion structure of the project: release phases, dependency ordering, and explicit decision thresholds for entering a new capability area.

Own the definition of what belongs in the MVP, v1, and post-v1 phases.

Own extension categories such as adapters, outputs, UI improvements, observability, packaging, and ecosystem integrations at the planning level.

Never own code paths, runtime behavior, or product analytics. Never mutate the core architecture without a new documented decision.

6. Inputs

The module consumes the project constitution documents, current architecture boundaries, MVP scope, and known user pain points from the core workflow.

It also consumes architectural constraints from prior module specifications, especially canonical trace model, analysis engine, postmortem generation, and UI/output concerns.

Inputs are not runtime data. They are policy inputs: capability gaps, maintenance limits, contributor bandwidth, and evidence from real usage.

7. Outputs

A phased roadmap structure, usually grouped into post-MVP horizons such as v1, v1.x, v2, and experimental tracks.

Decision criteria for admitting a future feature into a phase.

A prioritized set of extension themes with dependencies and blockers.

A deprecation and compatibility stance for future architectural change.

A written record of rejected ideas and why they remain out of scope.

8. Internal Design

Main subcomponents: roadmap horizons, feature gates, extension categories, dependency ledger, and review criteria.

Workflow: collect proposals -> classify by risk and dependency -> compare against project philosophy -> assign to horizon or reject -> publish with rationale.

Data flow is one-way. Ideas enter as candidate extensions and leave as approved future work, deferred work, or rejected work. No hidden “maybe later” state survives without rationale.

Important internal decision: keep the roadmap orthogonal to implementation modules. A roadmap item may reference a module, but it must not dictate internal code structure.

9. Data Model

Core entities are Extension Theme, Roadmap Item, Release Horizon, Dependency, Risk, and Decision Rationale.

An Extension Theme groups related work such as adapters, output formats, collaboration, or performance.

A Roadmap Item has a title, problem statement, user value, prerequisites, risk level, and exit criteria.

Relationships are many-to-one from Roadmap Item to Extension Theme and many-to-many between Roadmap Item and Dependencies.

This model exists so the roadmap can be reasoned about like an architecture artifact instead of a loose list of ideas.

10. Algorithms / Processing Logic

Step 1: classify each proposed extension by the type of value it adds, the subsystem it touches, and the risk it introduces.

Step 2: test the proposal against hard constraints: local-first default, deterministic analysis, framework agnosticism after normalization, and maintainability by a small team.

Step 3: determine whether the proposal is blocked by missing platform foundations, user evidence, or unresolved architectural debt.

Step 4: assign the proposal to a horizon only when its prerequisites are explicit and its scope is bounded.

Edge cases: some features are not roadmap items but platform bets. They should be marked as strategic research, not committed work.

Decision rule: when in doubt, defer. The cost of premature expansion is higher than the cost of waiting.

11. Tradeoffs

Simplicity vs flexibility: the roadmap must be simple enough to enforce, even if that means fewer categories and less ambition.

Generality vs specificity: generic “future work” language is useless. The document should be specific about the kind of extension and the dependency it needs.

Determinism vs heuristic inference: roadmap decisions should rely on explicit criteria, not vibes.

Why this design is preferred: it keeps expansion controlled and makes architectural drift visible before it becomes expensive.

12. Risks and Failure Modes

Hidden complexity: future work often hides infrastructure debt. Example: “add PDF export” can quietly require a whole rendering pipeline.

Ambiguous cases: some ideas look like UX improvements but actually imply a product platform. Those must be called out explicitly.

Unsupported inputs: speculative ideas without a user problem or dependency chain should not enter the roadmap as committed work.

Maintenance risks: a roadmap that is too broad becomes ceremonial and stops influencing actual decisions.

13. Testing Strategy

Unit tests are not the primary concern of this module. Its equivalent is review discipline: every roadmap item should be testable against documented criteria.

Integration checks should verify that roadmap decisions do not contradict module boundaries, release sequencing, or MVP constraints.

Fixture-driven review means reusing representative future proposals to confirm the decision framework produces consistent outcomes.

Regression protection comes from a decision log: if a similar feature was rejected before, the new proposal must explain why conditions changed.

14. Implementation Checklist

Define the next three roadmap horizons in advance: immediate post-MVP, v1 expansion, and strategic v2.

List all known extension themes and map each to prerequisites, user value, and risk.

Create explicit acceptance criteria for future roadmap entries.

Document at least one rejected example for each major theme so contributors understand the boundary.

Review roadmap items against the canonical trace model and analysis pipeline before public release.

15. Future Expansion

Natural next steps include broader adapter coverage, richer output formats, improved graphing, optional interactive UI features, and packaging improvements.

Later expansions may include telemetry, collaboration, and hosted workflows, but only after the local-first core remains robust and the project has a clear reason to externalize state.

Future roadmap updates should also consider governance for deprecation, compatibility windows, and long-term support policies.

16. Final Recommendation

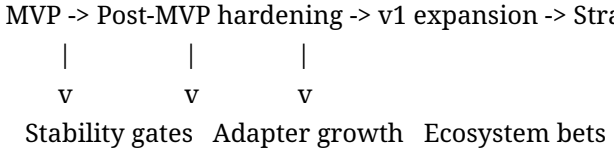
This module is MVP-ready as a policy artifact, not as a runtime subsystem.

The recommended simplification is to keep the initial roadmap narrow: one immediate post-MVP horizon, one medium-term horizon, and one speculative horizon. More than that is theater.

If the project later grows into multiple distribution channels or commercial offerings, this module should evolve into a formal strategy and lifecycle document with versioned roadmap reviews.

Roadmap Horizon Diagram

Text diagram:



Trace fidelity Output breadth Collaboration / hosted options

Feature Admission Criteria

Criterion	Question	Pass threshold	Failure action
User value	Does it solve a real debugging pain?	Clear evidence from target users	Defer or remove
Architecture fit	Does it preserve core boundaries?	No leakage into canonical analysis	Redesign scope
Maintainability	Can 2-3 engineers own it?	Small surface area and explicit dependencies	Split or defer
Determinism	Does it avoid hidden probabilistic behavior?	Yes, or isolated and optional	Reject for MVP-era

Final Notes

This module is the project's planning guardrail. It keeps the team honest about growth, prevents architecture drift, and makes future decisions legible to new contributors. If the roadmap ever starts resembling a feature buffet, this document should be the thing that says no.